Data Warehouse Project

Aurachiara Chirillo Student Id: 263800

University of Calabria

1 Data Source description

This project is based on the analysis of the "Brazilian E-Commerce Public Dataset by Olist" (available on Kaggle), a detailed collection containing information of around 100k orders from 2016 to 2018 made through Olist across major Brazilian marketplaces. Olist, which supplied the analyzed dataset, is an e-commerce platform connecting small businesses from all over Brazil to sales channels.

2 Preliminary Requirement Analysis

The data supplied by Olist has been converted into strategic insights. To guide the conceptual design and avoid including unnecessary data, a preliminary requirement analysis was performed. The analysis focuses on five core pillars:

- **Sales and Revenue Management**: The management needs to monitor the financial health of the marketplace, identifying growth trends and top-performing sectors. The aim is to answer questions such as:
  - "What is the total revenue (price) generated over time?"
  - "What is the total number of items sold over time?"
  - "How do the total revenue and the total number of items sold vary the state and the product category?"
- **Geo-Marketing**: It's important to Understand the revenue geographical distribution across the Brazil. The following questions must be answered:
  - "In which states are the most items purchased?"

  - "How is total revenue distributed across the different Brazilian states? Which states represent the core source of revenue?"

- **Delivery Efficiency Analysis**: possible delivery delays of the ordered items must be identified. The aim is to answer the following questions:
  - "What is the percentage of orders delivered on time, in advance and late analyzed by customer state?"
  - "What is the percentage of orders delivered on time or in advance analyzed by customer state?"

- **Shipping Cost Analysis**: Understanding the Average Shipping Cost by Seller State and understanding what percentage of the total cost incurred by the customer is attributable to shipping.
  - "What is the average shipping cost based on the seller's state?"
  - "What is the average shipping cost based on the seller's state and on product category?"

- **Logistics Cost Impact**: the question that needs to be answered is the following:
  - "How does the average percentage of shipping cost on the total cost paid by the customer vary depending on the destination state and product category?"

3 The Reconciled Database Schema

The original dataset, consisting of 9 files in CSV format, was reverse-engineered to obtain a relational model. This served to reduce redundancies and efficiently structure the data for generating the so-called **reconciled database** using PostgreSQL. From the analysis of the original dataset, the following relational model was derived:

Order(order_id, customer_id:Customer, order_status, order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date)
OrderItem(order_id:Order, order_item_id, product_id:Product, seller_id:Seller, shipping_limit_date, price, freight_value)
Customer(customer_id, customer_unique_id, customer_zip_code_prefix, customer_city,customer_state)

Product(product_id, product_category_name:ProductCategoryName, product_name_lenght, product_description_lenght, product_photos_qty, product_weight_g, product_length_cm, product_height_cm, product_width_cm)
Seller(seller_id, seller_zip_code_prefix, seller_city, seller_state)
Geolocation(geolocation_zip_code_prefix, geolocation_lat, geolocation_lng, geolocation_city, geolocation_state)
Payment(order_id:Order, payment_sequential, payment_type, payment_installments, payment_value)
Review(review_id, order_id:Order, review_score, review_comment_title, review_comment_message, review_creation_date, review_answer_timestamp)
ProductCategoryName(product_category_name, product_category_name_english)

Observation: The product_category_name column of the ProductCategoryName table and the homonymous column in the Product table contain the Portuguese names of the product categories.
The SQL script used to build the schema for the reconciled database using PostgreSQL is called Olist_reconciled_database_schema.sql and it is publicly available at the following repository:
https://github.com/Aurachiara9/data_warehouse.git

4 Data quality (DQ) assessment

In order to enable reliable analysis, it was crucial to assess the data quality and apply transformations steps. DQ Audit Checkpoints were performed to catch issues before they could propagate downstream. In particular, **Completeness**, **consistency**, **timeliness**, **validity** and **uniqueness** are the ISO 25012 DQ Dimensions to which a score between 0 and 1 has been assigned so to track quality trends over time. It's important to highlight that the operations executed are **repeatable** and **the scoring engine is deterministic and testable**.

The DQ was assessed for each useful table for the analysis (i.e. Order, OrderItem, Product, Customer, Seller and ProductCategoryName) considering only the relevant attributes and to do this validity and consistency rules were defined using Python. The columns not considered, as they were not used for the analysis, are:

product_name_lenght, product_description_lenght, product_photos_qty, product_weight_g, product_length_cm, product_height_cm, product_width_cm,

customer_unique_id, customer_zip_code_prefix,
seller_zip_code_prefix.

Table 1 summarizes the quality metrics computed on the raw dataset.

DQ SCORE COMPARISON

Table Dimension	Customer	Order	OrderItem	Product	ProductCategoryName	Seller
Completeness	1.000000	0.993831	1.000000	0.990744	1.000000	1.000000
Consistency	1.000000	0.981265	1.000000	1.000000	1.000000	1.000000
Timeliness	NaN	1.000000	NaN	NaN	NaN	NaN
Uniqueness	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
Validity	0.999990	1.000000	1.000000	0.281630	0.197183	0.992246

Table 1: Data Quality Summary

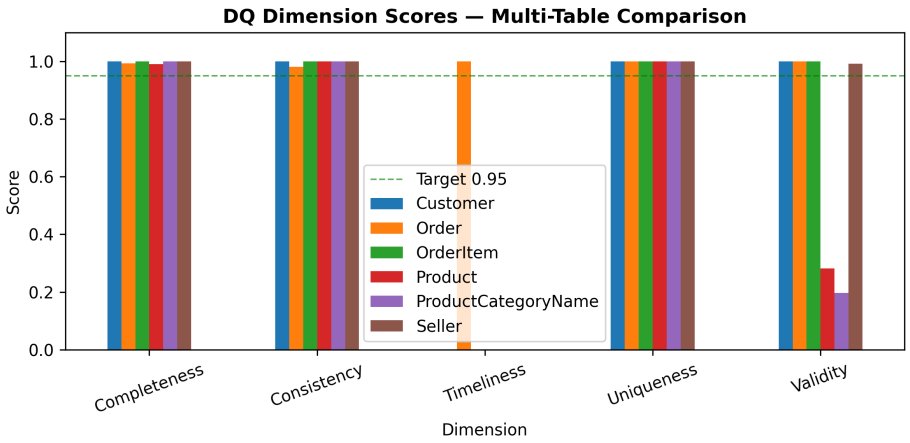


Figure 1: Data Quality Comparison

From the bar chart in Figure 1, we can see that all the analyzed dimensions (Completeness, Uniqueness, Consistency, and Timeliness), except for the values of Validity of the tables Product and ProductCategoryName, are above the Target (95%). The Uniqueness dimension has a maximum score of 1.0 (100%) across all tables. The Timeliness dimension, calculated only for the Order table, has a score of 1 and was calculated by checking whether the purchase date of the orders (order_purchase_timestamp) fell within the valid historical e-commerce range, set between 2016 and 2018. Since the data quality for almost all dimensions was already very high, the data cleaning and optimization efforts did not require radical changes to the dataset, but rather focused specifically on improving validity, completeness, and consistency.

For each table analyzed, a Missing Values Matrix and a horizontal bar chart were created to analyze the data quality dimensions under consideration. Below are the two charts generated based on the Order table.

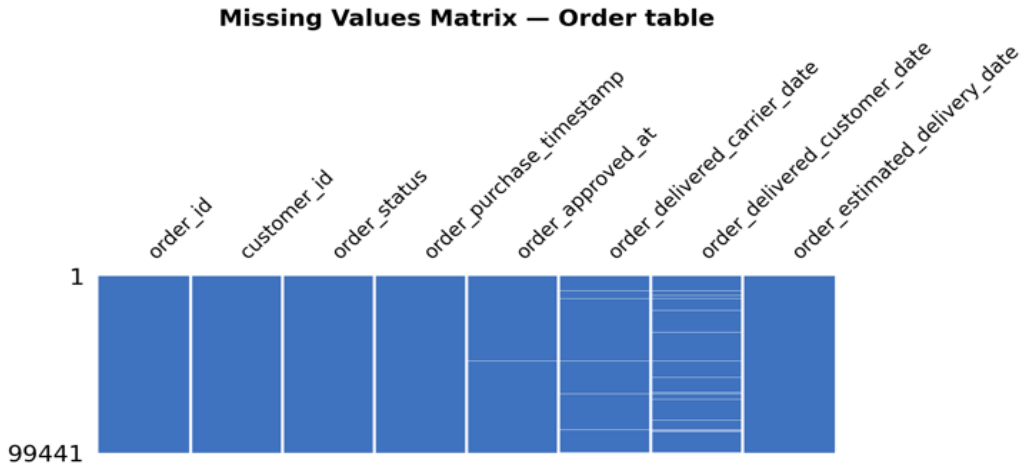


Figure 2: Missing values matrix of the Data Quality dimension scores for the Order table.

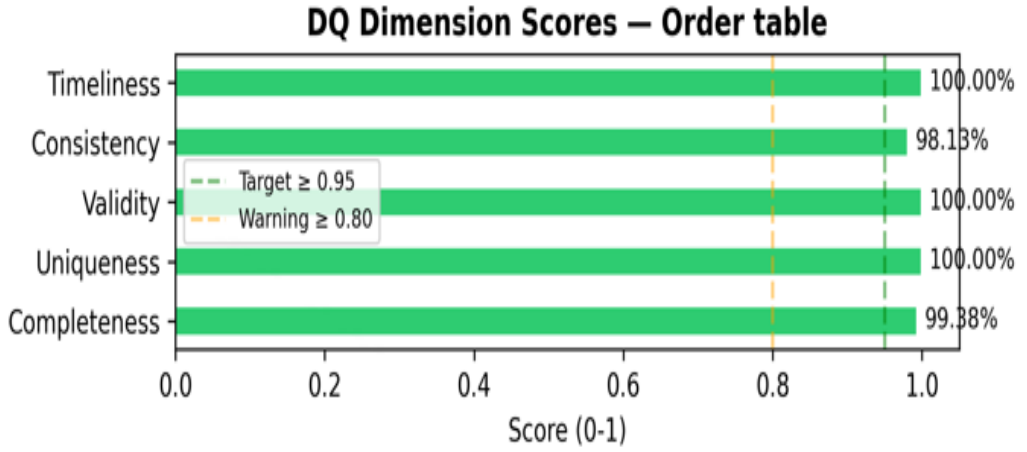


Figure 3: Horizontal bar chart of the Data Quality dimension scores for the Order table.

In a Missing Values Matrix, a column is completely blue if there are no

missing values, while any white rows indicate the presence of missing data. For the Order table:

- we can observe from the Missing Values Matrix that the columns order_approved_at, order_delivered_customer_date and order_estimated_delivery_date contain missing values.
- In the bar chart, consistency (98.13%) and completeness (99.38%) have the only scores slightly below 100%, but their scores are still excellent.

A further visual diagnostic assessment was performed for each relevant table:

- a horizontal bar chart displays the exact percentage of missing values exclusively for those attributes where missing data is actually present.
- for each column having values violating the validity rules, a chart is generated in which the total records of the table are segmented into 'Valid' and 'Invalid' categories, highlighting the exact number of rows that must be addressed during the cleaning phase.

To illustrate the output of this diagnostic layer, some results for the Order and Seller tables are discussed below.

Figure 4: Analysis of the Order table: horizontal bar chart of the missing values percentage.

As shown in Figure 3, the Order table contains a small percentage of missing values concentrated in three specific attributes: order_delivered_customer_date (3.0%), order_delivered_carrier_date (1.8%), and order_approved_at (0.15%).

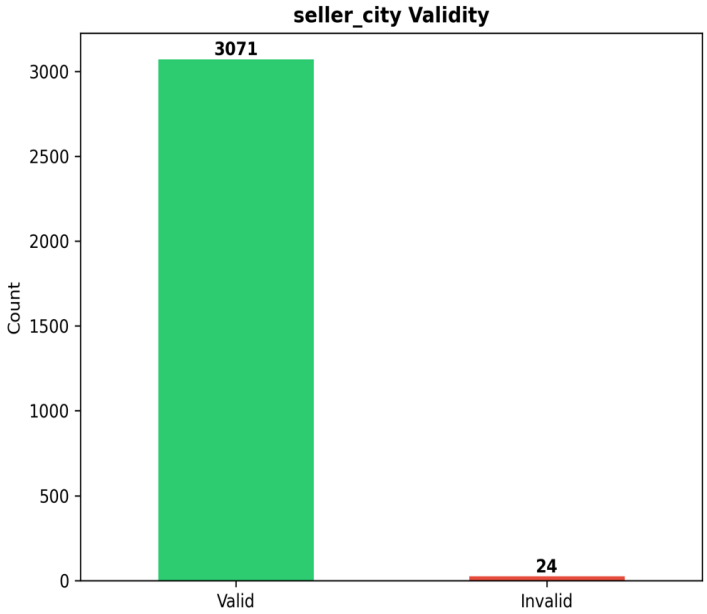


Figure 5: Analysis of the Seller table: distribution of valid and invalid values of the attribute seller_city.

Figure 4 highlights a validity issue regarding the attribute seller_city. Out of 3095 total sellers, 3071 records were flagged as valid (for all them the value of the attribute seller_city is valid), while 24 records were classified as invalid (each of these records has an invalid value of seller_city) .

4.1 LLM as a DQA Support Layer

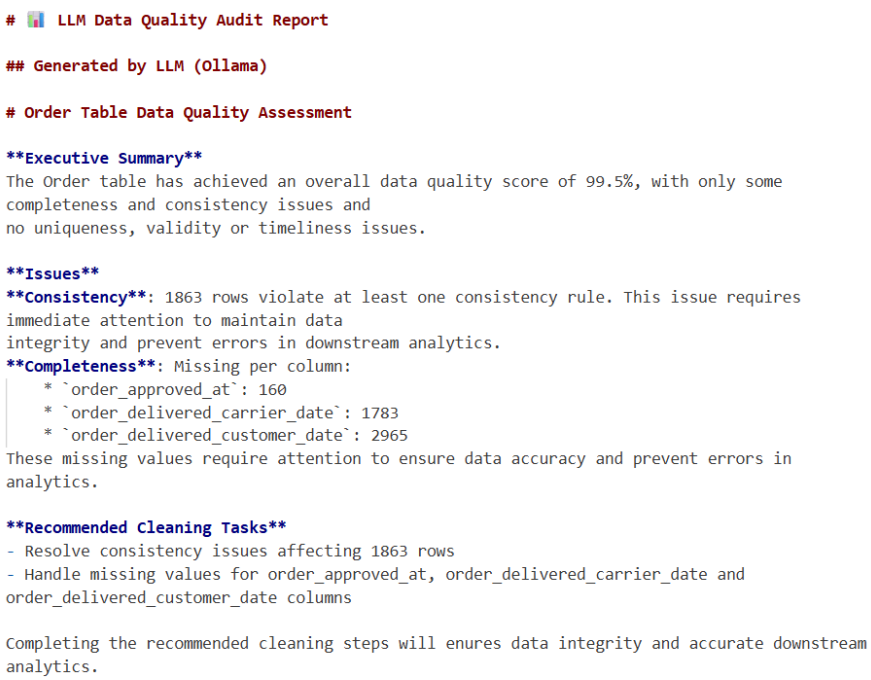
The **llama3.2:3b** model, provided by Ollama, was used to support the Data Quality assessment. The LLM was fed with the previously obtained DQ scorecards (containing the quality metrics computed) of the six considered tables so to get a professional Natural Language DQ Audit Report for each table. The following System Prompt was used to instruct the LLM to act as an expert Data Quality Analyst.

```
SYSTEM_AUDIT = """As an expert Data Quality Analyst with a focus on Data Warehousing, you will analyze a
structured DQ scorecard to generate clear and strategic audit reports for business stakeholders.
Your focus should be on identifying the issues and providing actionable recommendations based on their
potential business impact."""
```

Figure 6: System Prompt.

The generated reports have been carefully reviewed (i.e. human checks have been performed) and corrected where necessary. The Natural Language

DQ Audit Report for the Order table, after the **human checks and corrections**, is shown below as example:



```
# 📊 LLM Data Quality Audit Report

## Generated by LLM (Ollama)

# Order Table Data Quality Assessment

**Executive Summary**
The Order table has achieved an overall data quality score of 99.5%, with only some
completeness and consistency issues and
no uniqueness, validity or timeliness issues.

**Issues**
**Consistency**: 1863 rows violate at least one consistency rule. This issue requires
immediate attention to maintain data
integrity and prevent errors in downstream analytics.
**Completeness**: Missing per column:
    * `order_approved_at`: 160
    * `order_delivered_carrier_date`: 1783
    * `order_delivered_customer_date`: 2965
These missing values require attention to ensure data accuracy and prevent errors in
analytics.

**Recommended Cleaning Tasks**
- Resolve consistency issues affecting 1863 rows
- Handle missing values for order_approved_at, order_delivered_carrier_date and
order_delivered_customer_date columns

Completing the recommended cleaning steps will enures data integrity and accurate downstream
analytics.
```

Figure 7: Natural Language DQ Audit Report for the Order table.

5 Data Cleaning

In this section all the data cleaning operations applied are described. It's important to take into account that all these operations are documented in the Audit Log.

5.1 Missing Values

To analyze actual orders, we removed the rows from the Order table representing canceled orders (with order_status = canceled) and deleted the rows in OrderItem containing references to those orders. This significantly reduced the number of missing values.

The columns showing **missingness** were:

- **order_approved_at, order_delivered_carrier_date and order_delivered_customer_date** of the Order table. At the begin-

ning, they respectively had the 0.16%, 1.79% and 2.98% of missing values. It was applied a Chi-squared test of independence and the Logistic Regression for each of these columns and it was found their the Missing Value Mechanism is MAR (Missing At Random) since the missingness of these columns depends on the observed variable order_status. Missing values in these columns were handled using **group-wise mean imputation**. For each missing date, in each of the three columns considered, the value was estimated by calculating the average of the group of dates in the same column that have the same order status as the missing date.

- **product_category_name** of the Product table; in particular this column had the 1.85% of missing values. The joint analysis of Logistic Regression and Proxy Test reveals that the missing product category is not a random event (MCAR), but follows a structural logic linked to the economic and geographical characteristics of the dataset. For both statistical tests, the following three attributes were used simultaneously as predictors:

  - log_price: logarithmic transformation of the selling price.
  - log_freight: logarithmic transformation of the shipping cost.
  - seller_state: categorical variable representing the seller's geographic location.

  The choice to use the logarithmic versions (log_price, log_freight) instead of the raw values responds to specific technical needs: distribution normalization and variance stabilization.
  Since prices and shipping costs are recorded for each individual transaction (OrderItem table), the average price and the average freight_value was calculated for each product_id (i.e. for each product). This allowed to associate a representative and stable economic value with each product. This was essential for linking the different database tables.
  The Logistic Regression said that there is a weak MAR signal, i.e. Missingness is not strongly explained by observed variables; the proxy test said that product_category_name shows a possible MNAR pattern, indeed significant group differences were found.

After attempting to populate the dates, the few records that were still incomplete (i.e., those containing at least one null value in any column) were removed from the Order table and were therefore not considered for analysis. In the OrderItem table, only the items linked to orders that still exist were

retained.

Missing values in the Product table were handled as follows: records corresponding to products with a null value in the product_category_name column were removed from the Product table, and rows referring to those products were removed from the OrderItem table; orders that were left completely empty were then deleted from the Order table.

5.2 Outlier Detection

The numeric columns used for **outlier detection** are called **price** and **freight_value**.
It has been observed that the values in the "price" and "freight_value" columns of Olist are, in fact, highly skewed to the right. The Z-Score method was not applied since it would have assumed that the data follow a normal distribution and used the mean and standard deviation, but outliers "inflate" both these metrics so leading extreme values ending up being "masked" and making the test less effective at detecting them. For this reason, instead of using the Z-score method, the modified Z-score method was used. The latter is superior because it uses the median and the MAD (Median Absolute Deviation), which are not influenced by outliers.

To identify outliers in the price and freight_value columns the IQR Fence method and the Isolation Forest method were also used. Like the Modified Z-Score, these two methods do not assume that the data follow a normal distribution and are not based on mean and standard deviation.
The IQR Fence method, based on quartiles, is suitable for analyzing unbalanced data.
The Isolation Forest method attempts to understand how easy it is to isolate a point. Outliers, located in low-density regions and having extreme values, are isolated very quickly. Normal data are located in dense regions and require many more cuts to isolate.
No single method is sufficient for outlier detection. Combining multiple methods and using a consensus threshold (i.e. a row is flagged as a 'consensus outlier' only if it is identified as anomalous by a minimum number of methods) provide more reliable results. The code implements a threshold of ≥ 2 methods.

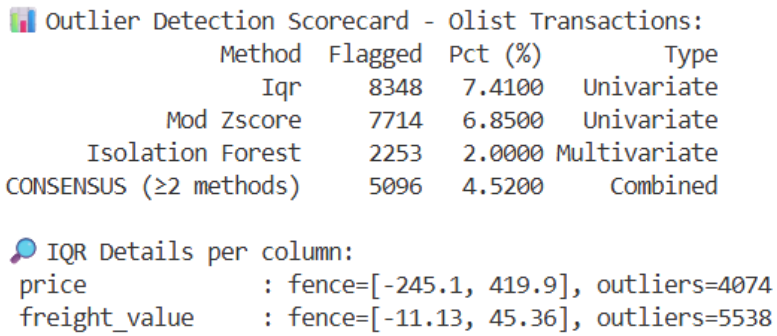


Figure 8: Outlier Detection results

The anomalies identified by consensus, as we can observe in Figure 6, are 5096; the strategy applied to these anomalies was the **winsorization**.

5.3 Invalid Values

The following invalid values were found:

- 24 values in the column seller_city of the Seller table. For each of these values, a cleanup sequence was applied, e.g., information in parentheses was removed and any spaces at the beginning and end were removed. If the value changed and was found to be valid, the DataFrame was updated with the new name. Only two values were still invalid after this cleanup: vendas@creditparts.com.br and *04482255*. The sellers associated to these two values have been deleted: the impact of this operation was minimal since the removed sellers were tied to very few orders. In addition, from the sold items table (i.e., the OrderItem table), records associated with such removed vendors were deleted and, from the Order table, all orders that were left empty as a result of the removal of those records were deleted.

- 1 value in the column customer_city of the Customer table. Since there were initially very many orders and only one order had a customer with an invalid customer_city and that order corresponded to only one row in the OrderItem table, it was decided to delete that order, the customer and the associated order item.

- 23671 values in the column product_category_name in the table Product. The invalid categories were analyzed and their similarity to the correct terms (using an 80% similarity threshold) in the ProductCategoryName table was calculated. If an invalid value was found to be

very close to an official one in the ProductCategoryName table, it was replaced with the official one: this approach handled 11 invalid values. Invalid values that were not similar to the correct ones but had multiple occurrences were added to the official dictionary: in this way 2 invalid values have been handled. Products with isolated incorrect categories (a single occurrence) were removed. To maintain database integrity, removing a product automatically deleted its OrderItems and any empty Orders.

- 54 invalid values in the column product_category_name and 56 invalid values in the column product_category_name_english of the table ProductCategoryName.

For simplicity and clarity in reading, only names of categories without underscore were considered valid: in names of categories containing underscores, each underscore was replaced by a space character so to have valid names.
Names of categories containing underscores constituted almost all the invalid values in the Product table and all the invalid values in the ProductCategoryName table.
Besides, there were two product_category_name which appeared both with and without the number 2 (eletrodomesticos and eletrodomesticos_2, casa_conforto and casa_conforto_2). Since there were not a clear reason (i.e. there were no clear differences of weight/volume/price) for the presence of the 2, For the products which at the beginning were of a category whose name contained a number, in the category name was removed the number.

5.4 Handling some temporal inconsistencies in the Order table

To avoid any issues with the display, a very small percentage of orders (about 0.17%) have been removed since they had a carrier delivery date (order_delivered_carrier_date) prior to the purchase date (order_purchase_timestamp). The rows referring to these removed orders were deleted from the OrderItem table to ensure referential integrity.

In cases where the order was delivered to the carrier (order_delivered_carrier_date) before it was formally approved (order_approved_at), the approval date was set to match the delivery date to the carrier.

For records in the Order table where the customer was shown to have received the package (order_delivered_customer_date) before it was delivered

to the carrier (order_delivered_carrier_date), the delivery date to the customer was moved to one hour after the shipping date.

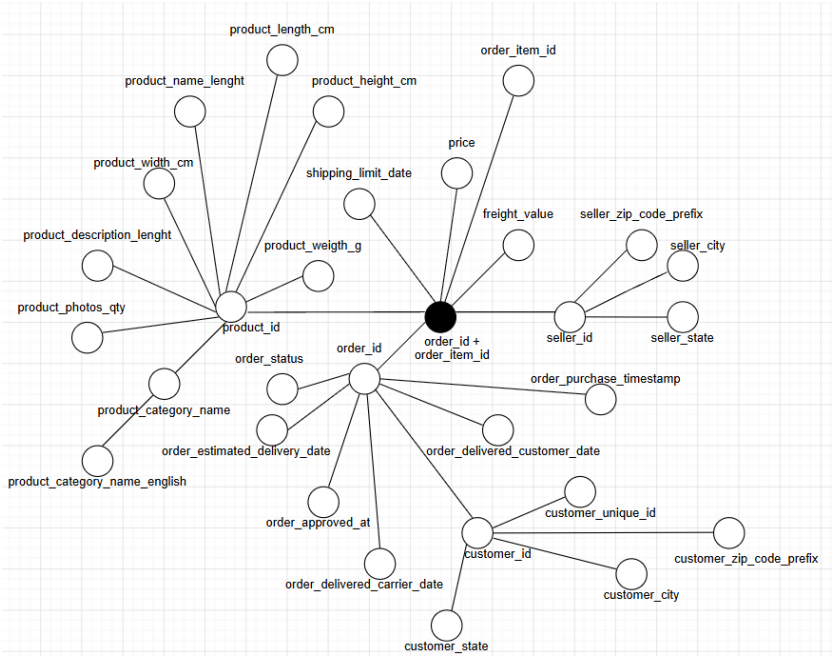
Cases where the estimated delivery date (order_estimated_delivery_date) was earlier than the actual shipment date (order_delivered_carrier_date) have been corrected: the estimated delivery date has been postponed to 1 day after the shipment date.

6 The conceptual design

The table that represents the fact of interest is OrderItem, as it models the fundamental business event: the sale of a single product by a given seller within an order.

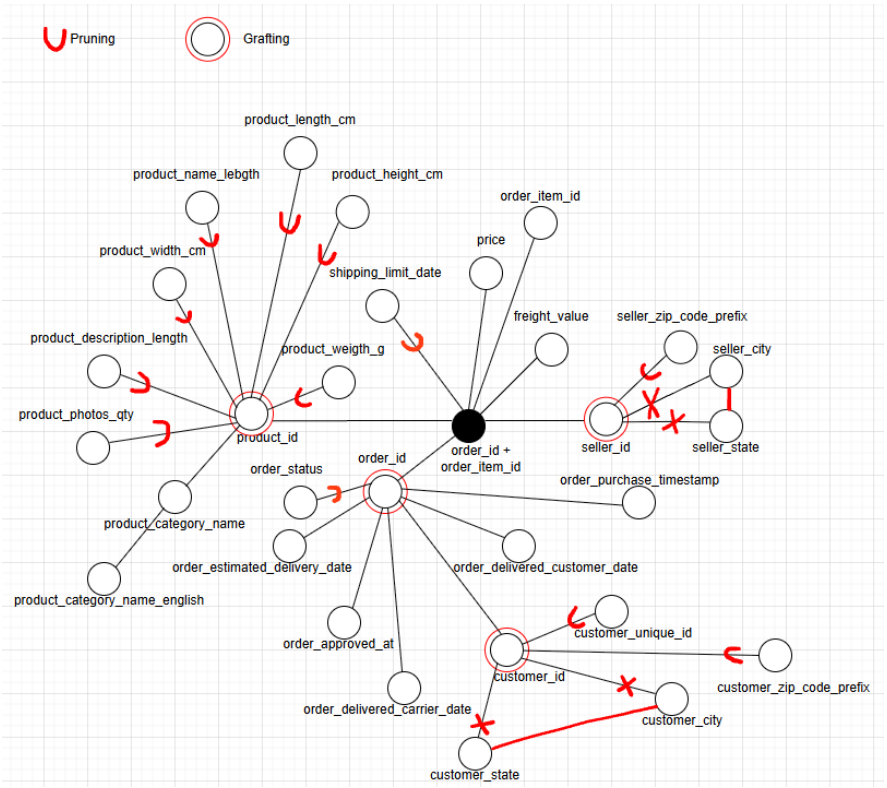
6.1 Attribute Tree

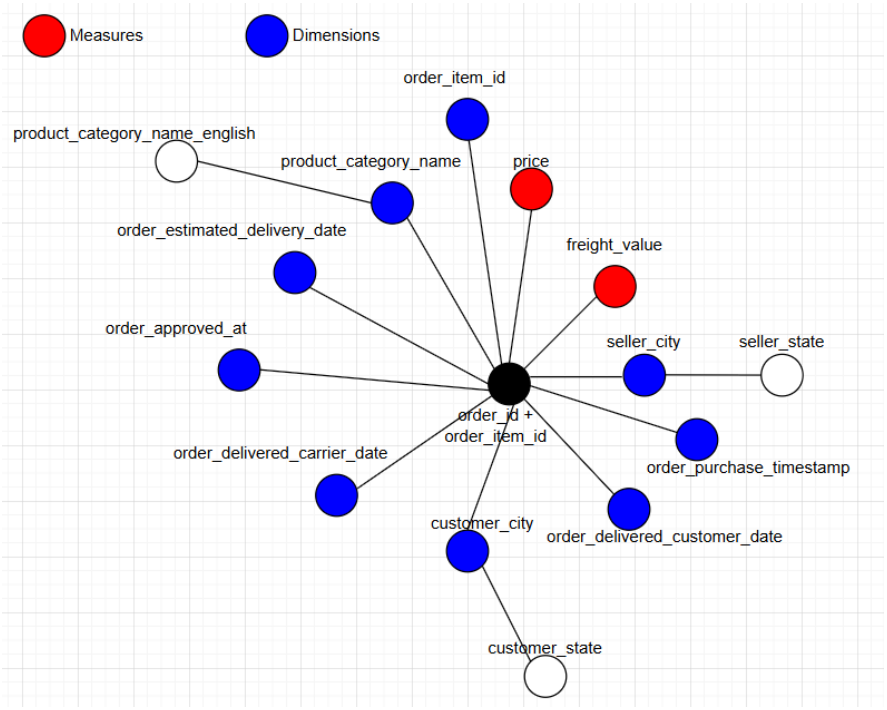
The construction of the attribute tree allowed us to identify the dimensions and measures relevant for the definition of the Fact Schema



The goal of the **editing phase** is to clean up the attribute tree to keep only what is useful for multidimensional analysis.

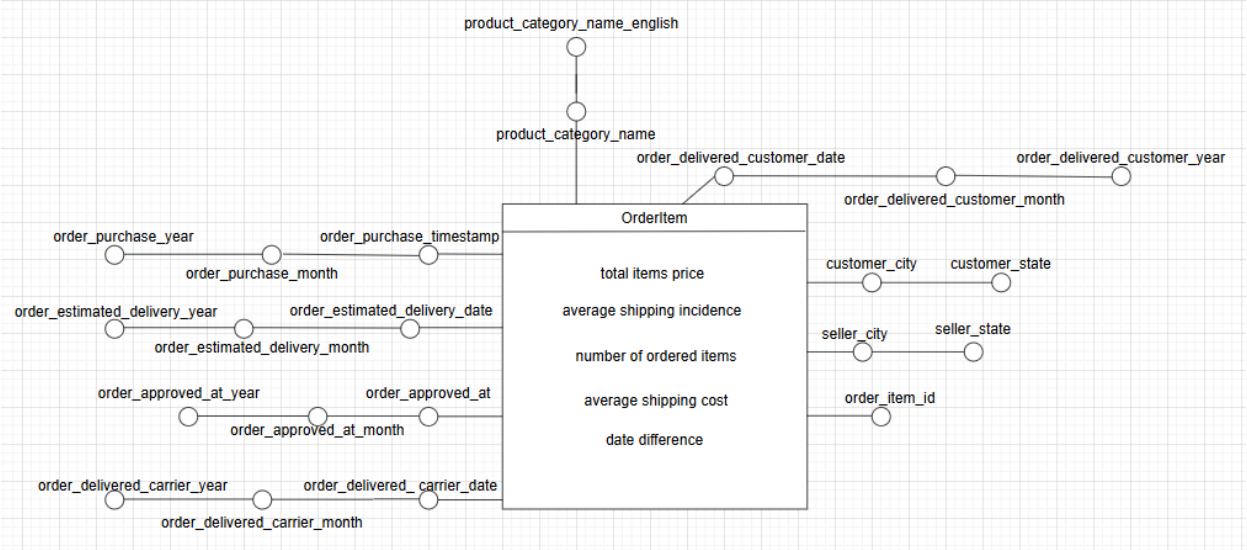
Pruning: Vertices that were irrelevant to the analysis performed in this project were removed. These vertices were the following: shipping_limit_date, product_weight_g, product_height_cm, product_length_cm, product_name_length, product_width_cm, product_description_length, product_photos_qty, order_status, customer_unique_id, seller_zip_code_prefix, customer_zip_code_prefix.
Grafting: Grafting was applied to vertices deemed irrelevant to the analysis, but which have at least one descendant containing information useful for the study. In particular, grafting was applied to the following vertices: product_id, order_id, customer_id, seller_id.

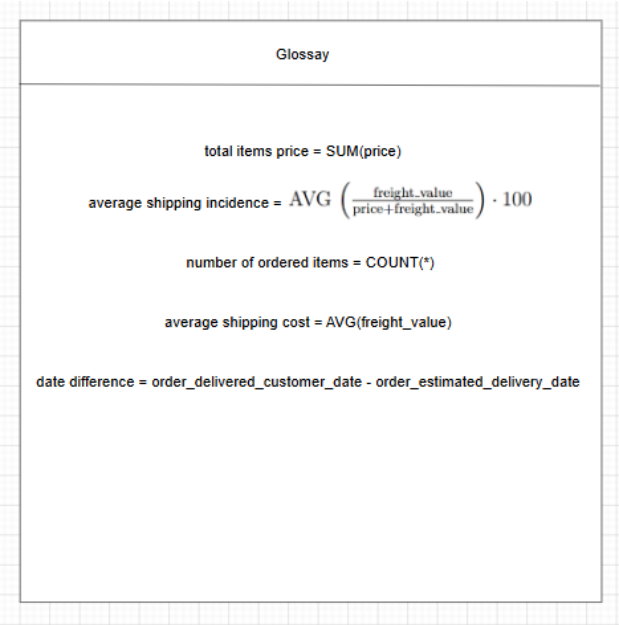

Pruning
Grafting
product_length_cm
product_name_lebgth
product_height_cm
order_item_id
price
product_width_cm
shipping_limit_date
freight_value
seller_zip_code_prefix
seller_city
product_description_length
product_weigth_g
product_photos_qty
product_id
order_id +
order_item_id
seller_id
seller_state
order_status
order_id
order_purchase_timestamp
product_category_name
order_estimated_delivery_date
order_delivered_customer_date
product_category_name_english
order_approved_at
customer_unique_id
customer_id
customer_zip_code_prefix
order_delivered_carrier_date
customer_city
customer_state


Measures
Dimensions
order_item_id
product_category_name_english
product_category_name
price
order_estimated_delivery_date
freight_value
order_approved_at
seller_city
seller_state
order_id +
order_item_id
order_delivered_carrier_date
order_purchase_timestamp
customer_city
order_delivered_customer_date
customer_state

7 Fact Schema

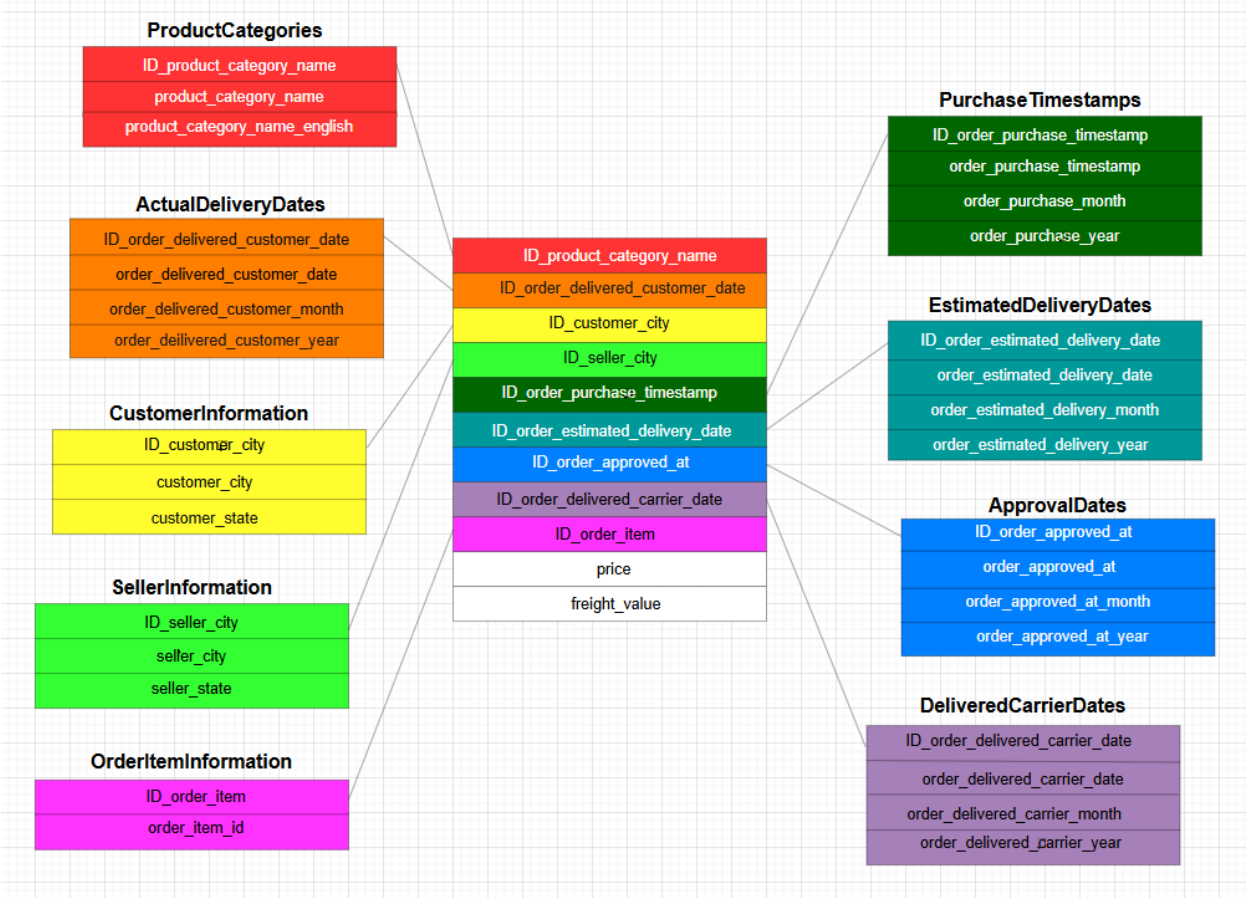
After the pruning and grafting operations, the Attribute Tree was converted into the following Fact Schema, showing the relevant fact of interest.





8 Star Schema

Starting from the fact schema illustrated before, the following star schema was designed:



The file Olist_data_warehouse_schema.sql , containing the code to build the schema of the star schema for the data warehouse, and all the Python files used for the "PHASE 2: DATA MANAGEMENT" are publicly available at the following repository:
https://github.com/Aurachiara9/data_warehouse.git